

**LAPORAN FINAL PROJECT STRUKTUR DATA
SISTEM ANTRIAN LAYANAN MENGGUNAKAN QUEUE DAN STACK**



Disusun Oleh
MUHAMMAD ZAKY MAULANA
2215061070

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS LAMPUNG
BANDAR LAMPUNG
2026

DAFTAR ISI

	Halaman
DAFTAR ISI.....	ii
DAFTAR GAMBAR	v
DAFTAR TABEL	vii
1. PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah.....	2
1.4 Tujuan Project	3
II. DASAR TEORI	4
2.1 Struktur Data	4
2.2 Node dan Linked List.....	5
2.3 Queue	5
2.4 Stack.....	6
2.5 Linear Search	7
2.6 Traversal.....	7
2.7 Kompleksitas Algoritma	8

III.	PERANCANGAN SISTEM	10
3.1	Deskripsi Sistem	10
3.2	Flowchart Sistem.....	11
3.3	Desain Struktur Data	14
3.4	Desain Algoritma	16
IV.	IMPLEMENTASI	19
4.1	Implementasi Program	19
4.2	. Implementasi Node	19
4.3	Implementasi Queue.....	20
4.4	Implementasi Enqueue	22
4.5	Implementasi Dequeue.....	23
4.6	Implementasi Tampilkan Antrian Aktif.....	24
4.7	Implementasi Stack	25
4.8	Implementasi Push pada Stack.....	26
4.9	Implementasi Tampilkan Riwayat.....	27
4.10	Implementasi Searching	28
4.11	Implementasi Menu Utama	30
4.12	Implementasi Menu Tambah Antrian	31
4.13	Implementasi Menu Panggil Antrian	33
4.14	Implementasi Menu Cari Antrian	34
4.15	Implementasi Menu Tampilkan Data dan Keluar	34
V.	KESIMPULAN.....	36
5.1	Hasil Akhir	36
5.2	Kekurangan	36
5.3	Saran Pengembangan	37

5.4	Kesimpulan Umum	37
DAFTAR PUSTAKA.....		38

DAFTAR GAMBAR

	Halaman
Gambar 1. Struktur Data	4
Gambar 2. Queue	5
Gambar 3. Stack	6
Gambar 4. Linear Search.....	7
Gambar 5. Traversal.....	7
Gambar 6. Flowchart Project	11
Gambar 7. Implementasi Node pada C++.....	20
Gambar 8. Implementasi Node pada Python.....	20
Gambar 9. Implementasi Class Queue pada C++	21
Gambar 10. Implementasi Class Queue pada Python	21
Gambar 11. Implementasi Fungsi Enqueue pada C++.....	22
Gambar 12. Implementasi Fungsi Enqueue pada Python	22
Gambar 13. Implementasi Dequeue pada C++	23
Gambar 14. Implementasi Dequeue pada Python	23
Gambar 15. Implementasi Tampilkan Antrian Aktif pada C++	24
Gambar 16. Implementasi Tampilkan Antrian Aktif pada Python	25
Gambar 17. Implementasi Stack pada C++	26
Gambar 18. Implementasi Stack pada Python	26
Gambar 19. Implementasi Fungsi Push pada C++.....	27

Gambar 20. Implementasi Fungsi Push pada Python	27
Gambar 21. Implementasi Fungsi Tampilkan Riwayat pada C++	28
Gambar 22. Implementasi Fungsi Tampilkan Riwayat pada Python	28
Gambar 23 Implementasi Searching pada C++	29
Gambar 24. Implementasi Searching pada Python	29
Gambar 25. Implementasi Menu Utama pada C++	31
Gambar 26. Implementasi Menu Utama pada Python	31
Gambar 27. Implementasi Menu Tambah Antrian pada C++	32
Gambar 28. Implementasi Menu Tambah Antrian pada Python	32
Gambar 29. Implementasi Menu Panggil Antrian pada C++	33
Gambar 30. Implementasi Menu Panggil Antrian pada Python.....	33
Gambar 31. Implementasi Menu Cari Antrian pada C++	34
Gambar 32. Implementasi Menu Cari Antrian pada Python	34
Gambar 33. Implementasi Menu Tampilkan Data dan Keluar pada C++	35
Gambar 34. Implementasi Menu Tampilkan Data dan Keluar pada Python.....	35

DAFTAR TABEL

	Halaman
Tabel 1. Kebutuhan Sistem	10
Tabel 2. Desain Struktur Data	15
Tabel 3. Desain Algoritma.....	16

1. PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu komputer karena berhubungan dengan cara data disimpan, diatur, dan diproses agar program dapat berjalan dengan lebih efektif. Pemilihan struktur data yang tepat akan membantu program menyelesaikan masalah dengan logika yang lebih jelas dan penggunaan sumber daya yang lebih efisien.

Salah satu permasalahan nyata yang dapat diselesaikan menggunakan struktur data adalah sistem antrian layanan. Dalam kehidupan sehari-hari, sistem antrian dapat ditemukan pada loket pelayanan, rumah sakit, bank, administrasi kampus, maupun layanan pelanggan. Pada sistem seperti ini, data yang pertama masuk harus diproses terlebih dahulu. Oleh karena itu, struktur data Queue cocok digunakan karena memiliki prinsip First In First Out atau FIFO.

Selain antrian aktif, sistem juga membutuhkan pencatatan riwayat data yang sudah dipanggil. Untuk kebutuhan tersebut, digunakan struktur data Stack. Stack menyimpan data berdasarkan prinsip Last In First Out atau LIFO, sehingga data yang terakhir dipanggil dapat ditampilkan terlebih dahulu pada daftar riwayat.

Berdasarkan kebutuhan tersebut, project ini mengembangkan program Sistem Antrian Layanan menggunakan dua bahasa pemrograman, yaitu C++ dan Python. Program ini mengimplementasikan Queue dan Stack secara manual menggunakan konsep node atau linked list, tanpa menggunakan library struktur data siap pakai.

1.2 Rumusan Masalah

Berdasarkan uraian pada latar belakang, rumusan masalah dalam project ini adalah sebagai berikut:

- 1) Bagaimana merancang sistem antrian layanan dengan mengimplementasikan struktur data Queue untuk mengelola antrian aktif dan Stack untuk menyimpan riwayat antrian serta menerapkan algoritma searching untuk mencari data antrian berdasarkan nomor antrian?
- 2) Bagaimana menganalisis kompleksitas waktu dan memori dari fitur-fitur program?

1.3 Batasan Masalah

Batasan masalah project ini yaitu sebagai berikut:

- 1) Program dibuat berbasis terminal, bukan berbasis GUI atau web.
- 2) Data antrian hanya tersimpan selama program berjalan dan belum menggunakan database.
- 3) Struktur data Queue dan Stack dibuat secara manual menggunakan node.
- 4) Program tidak menggunakan library struktur data siap pakai seperti queue, stack, list eksternal, atau collections.deque.

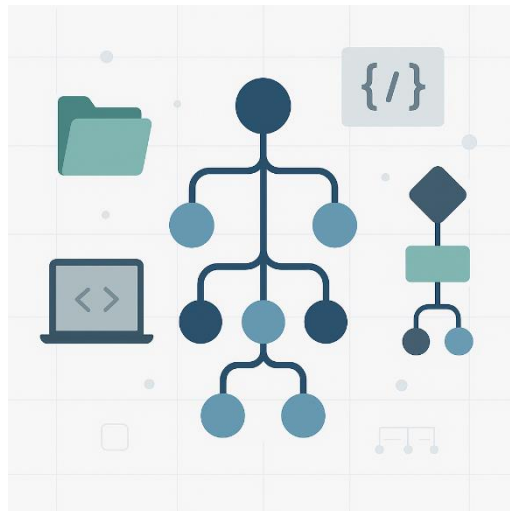
1.4 Tujuan Project

Berdasarkan rumusan masalah yang telah dipaparkan, maka tujuan perancangan project ini adalah sebagai berikut.

- 1) Merancang sistem antrian layanan dengan mengimplementasikan struktur data Queue untuk mengelola antrian aktif dan Stack untuk menyimpan riwayat antrian serta menerapkan algoritma searching untuk mencari data antrian berdasarkan nomor antrian.
- 2) Menganalisis kompleksitas waktu dan memori dari fitur-fitur program.

II. DASAR TEORI

2.1 Struktur Data



Gambar 1. Struktur Data

Struktur data adalah cara untuk menyimpan dan mengorganisasi data di dalam program agar data tersebut dapat digunakan secara efisien. Dalam project ini, struktur data digunakan untuk mengelola data antrian layanan, mulai dari data yang masih menunggu dipanggil hingga data yang sudah selesai dipanggil.

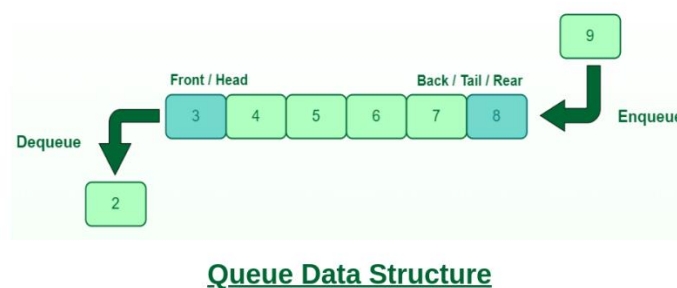
Struktur data yang digunakan pada project ini adalah Queue, Stack, dan Node/Linked List. Queue digunakan untuk antrian aktif, Stack digunakan untuk riwayat panggilan, sedangkan Node/Linked List digunakan sebagai dasar implementasi manual dari kedua struktur data tersebut.

2.2 Node dan Linked List

Node adalah unit data yang menyimpan nilai dan alamat atau referensi menuju node berikutnya. Pada project ini, setiap node menyimpan nomor antrian, nama, keperluan, dan pointer next. Kumpulan node yang saling terhubung membentuk linked list.

Linked list cocok digunakan karena ukuran data dapat berubah secara dinamis saat program berjalan. Morin menjelaskan bahwa linked list tersusun dari node yang menyimpan data dan referensi ke node berikutnya. Pada singly linked list, operasi Stack dan FIFO Queue dapat dilakukan secara efisien menggunakan head dan tail. Referensi dasar teori ini menggunakan Morin (2013), Open Data Structures, halaman buku 63-66.

2.3 Queue



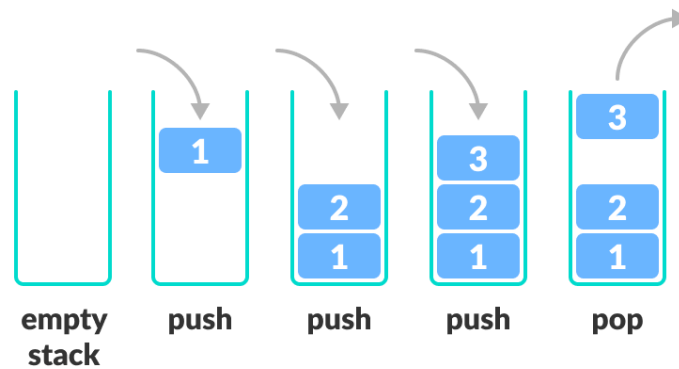
Gambar 2. Queue

Queue adalah struktur data linear yang menggunakan prinsip First In First Out atau FIFO. Artinya, data yang pertama kali masuk ke dalam queue akan menjadi data pertama yang keluar. Prinsip ini sesuai dengan sistem antrian layanan, karena pelanggan atau pengguna yang mengambil nomor antrian lebih awal harus dilayani lebih dulu.

Operasi utama pada Queue adalah enqueue dan dequeue. Enqueue digunakan untuk menambahkan data baru ke bagian belakang antrian,

sedangkan dequeue digunakan untuk mengambil atau menghapus data dari bagian depan antrian. Pada project ini, operasi enqueue dilakukan melalui pointer rear, sedangkan operasi dequeue dilakukan melalui pointer front.

2.4 Stack

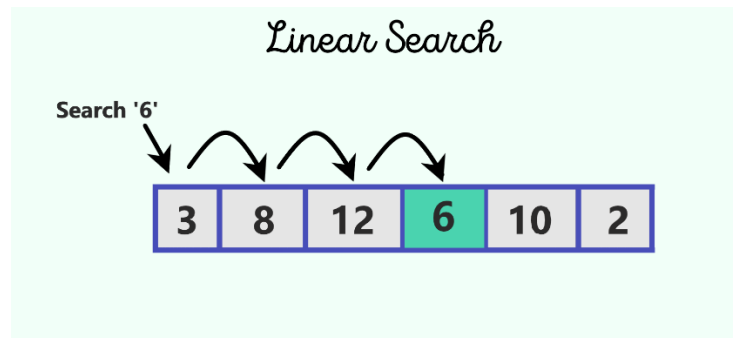


Gambar 3. Stack

Stack adalah struktur data linear yang menggunakan prinsip Last In First Out atau LIFO. Artinya, data yang terakhir masuk akan menjadi data pertama yang keluar. Pada project ini, Stack digunakan untuk menyimpan riwayat antrian yang sudah dipanggil.

Operasi utama pada Stack adalah push dan pop. Namun, pada project ini operasi yang digunakan adalah push dan traversal riwayat. Push digunakan untuk memasukkan data antrian yang sudah dipanggil ke bagian atas Stack. Riwayat kemudian ditampilkan dari data paling baru ke data yang lebih lama.

2.5 Linear Search

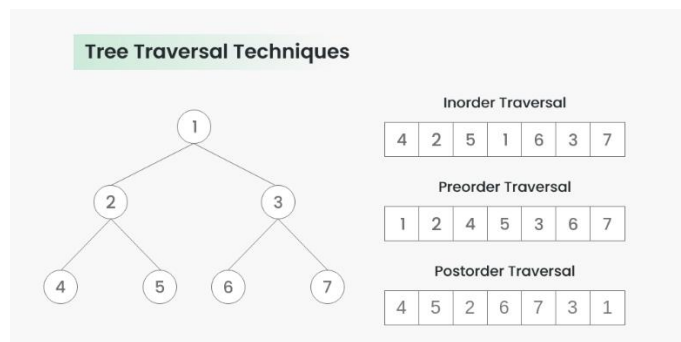


Gambar 4. Linear Search

Linear search adalah algoritma pencarian yang bekerja dengan cara memeriksa data satu per satu dari awal sampai akhir hingga data yang dicari ditemukan. Jika data ditemukan, program akan menampilkan informasi data tersebut. Jika tidak ditemukan sampai akhir struktur data, program akan menampilkan pesan bahwa data tidak tersedia.

Pada project ini, linear search digunakan untuk mencari nomor antrian pada Queue. Pencarian dilakukan mulai dari node front, kemudian berpindah ke node berikutnya melalui pointer next. Morin (2013) juga menjelaskan pencarian linear pada list sebagai proses memeriksa elemen satu per satu hingga elemen ditemukan. Referensi ini terdapat pada Open Data Structures, halaman buku 108-109.

2.6 Traversal



Gambar 5. Traversal

Traversal adalah proses mengunjungi setiap elemen dalam struktur data. Pada project ini, traversal digunakan untuk menampilkan seluruh data antrian aktif dan seluruh riwayat panggilan. Traversal dilakukan dengan memulai dari node pertama, kemudian bergerak ke node berikutnya sampai pointer bernilai null.

2.7 Kompleksitas Algoritma

Kompleksitas algoritma digunakan untuk menganalisis tingkat efisiensi suatu operasi berdasarkan jumlah data yang diproses. Kompleksitas waktu menunjukkan banyaknya langkah atau waktu yang dibutuhkan program dalam menjalankan suatu operasi, sedangkan kompleksitas memori menunjukkan ruang penyimpanan yang digunakan selama program berjalan.

Menurut Morin (2013), analisis terhadap struktur data dapat dilihat dari beberapa aspek utama, yaitu kebenaran operasi, kompleksitas waktu, dan kompleksitas ruang. Kompleksitas waktu berkaitan dengan waktu eksekusi operasi pada struktur data, sedangkan kompleksitas ruang berkaitan dengan jumlah memori yang digunakan oleh struktur data. Selain itu, Morin juga menjelaskan bahwa analisis waktu operasi umumnya menggunakan notasi Big-O karena perhitungan waktu eksekusi secara tepat sulit dilakukan dan dapat berbeda pada setiap komputer.

Pada project ini, operasi enqueue, dequeue, dan push memiliki kompleksitas waktu $O(1)$ karena prosesnya hanya melakukan perubahan pointer pada bagian tertentu, tanpa harus menelusuri seluruh data. Operasi enqueue menambahkan data pada bagian belakang queue melalui pointer rear, operasi dequeue mengambil data dari bagian depan queue melalui pointer front, sedangkan operasi push menambahkan data pada bagian atas stack melalui pointer top.

Sementara itu, operasi searching dan traversal memiliki kompleksitas waktu $O(n)$. Hal ini disebabkan karena program harus menelusuri data satu per satu dari node awal sampai node terakhir atau sampai data yang dicari ditemukan. Dengan demikian, semakin banyak jumlah data dalam antrian, maka semakin banyak pula kemungkinan langkah yang perlu dilakukan oleh program. Referensi ini menggunakan Morin (2013), *Open Data Structures: An Introduction*, halaman buku 12–14 untuk pembahasan notasi Big-O dan halaman buku 19–20 untuk pembahasan time complexity serta space complexity.

III. PERANCANGAN SISTEM

3.1 Deskripsi Sistem

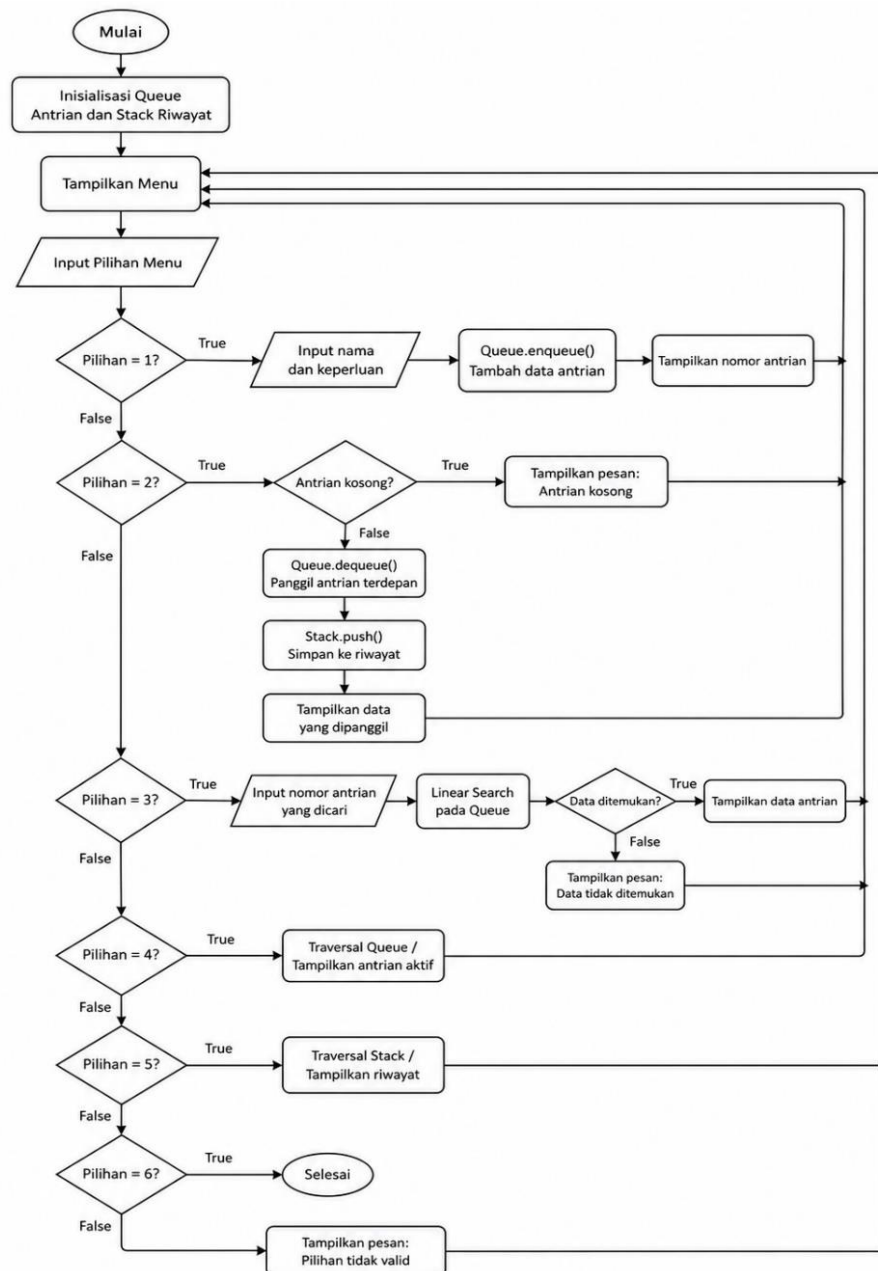
Sistem Antrian Layanan adalah program berbasis terminal yang digunakan untuk mengelola data antrian. Program menyediakan menu untuk menambah antrian, memanggil antrian, mencari data antrian, menampilkan antrian aktif, menampilkan riwayat panggilan, dan keluar dari program.

Program menggunakan nomor antrian otomatis. Setiap pengguna yang ditambahkan akan memperoleh nomor antrian berdasarkan urutan masuk. Ketika antrian dipanggil, data terdepan pada Queue akan dikeluarkan dan dipindahkan ke Stack sebagai riwayat panggilan.

Tabel 1. Kebutuhan Sistem

No	Kebutuhan	Keterangan
1	Bahasa Pemrograman	C++ dan Python
2	Media Eksekusi	Terminal/CMD
3	Struktur Data	Queue, Stack, dan Node/Linked List
4	Algoritma	Linear Search dan Traversal
5	Input Data	Nama dan keperluan pengguna
6	Output Data	Nomor antrian, daftar antrian, hasil pencarian, dan riwayat panggilan

3.2 Flowchart Sistem



Gambar 6. Flowchart Project

Flowchart sistem antrian layanan menggambarkan alur kerja program secara keseluruhan, mulai dari tahap awal program dijalankan hingga program dihentikan oleh pengguna. Sistem ini dirancang dalam bentuk program berbasis menu, sehingga pengguna dapat memilih beberapa fitur

yang tersedia sesuai kebutuhan. Setiap proses yang telah selesai dijalankan akan diarahkan kembali ke menu utama, kecuali apabila pengguna memilih menu keluar.

Alur program diawali dengan simbol Mulai, kemudian dilanjutkan dengan proses inisialisasi Queue Antrian dan Stack Riwayat. Pada tahap ini, sistem menyiapkan struktur data yang akan digunakan dalam program. Struktur data queue digunakan untuk menyimpan data antrian aktif, sedangkan struktur data stack digunakan untuk menyimpan riwayat data antrian yang telah dipanggil.

Setelah proses inisialisasi selesai, sistem akan menampilkan menu utama kepada pengguna. Pengguna kemudian diminta untuk memasukkan pilihan menu. Input pilihan tersebut akan diperiksa secara berurutan menggunakan percabangan kondisi. Pemeriksaan dilakukan mulai dari pilihan menu 1 sampai dengan pilihan menu 6. Apabila suatu kondisi bernilai benar, maka sistem akan menjalankan proses sesuai menu yang dipilih. Namun, apabila kondisi bernilai salah, maka sistem akan melanjutkan pemeriksaan ke pilihan menu berikutnya.

Pada pilihan menu pertama, yaitu Tambah Antrian, sistem meminta pengguna untuk memasukkan nama dan keperluan. Data tersebut kemudian diproses menggunakan operasi enqueue pada struktur data queue. Operasi ini berfungsi untuk menambahkan data baru ke bagian belakang antrian. Setelah data berhasil ditambahkan, sistem akan menampilkan nomor antrian kepada pengguna, kemudian alur program kembali ke menu utama.

Pada pilihan menu kedua, yaitu Panggil Antrian, sistem terlebih dahulu memeriksa apakah antrian dalam keadaan kosong. Jika antrian kosong, maka sistem akan menampilkan pesan bahwa antrian tidak tersedia. Namun, jika antrian tidak kosong, sistem akan menjalankan operasi dequeue

untuk mengambil data antrian yang berada di posisi paling depan. Data yang telah dipanggil kemudian disimpan ke dalam stack riwayat menggunakan operasi push. Setelah itu, sistem menampilkan data antrian yang dipanggil dan kembali ke menu utama.

Pada pilihan menu ketiga, yaitu Cari Antrian, pengguna diminta untuk memasukkan nomor antrian yang ingin dicari. Sistem kemudian melakukan proses pencarian menggunakan algoritma linear search pada data yang terdapat di dalam queue. Jika data ditemukan, maka sistem akan menampilkan informasi antrian yang sesuai. Sebaliknya, jika data tidak ditemukan, sistem akan menampilkan pesan bahwa data tidak tersedia. Setelah proses pencarian selesai, alur program kembali ke menu utama.

Pada pilihan menu keempat, yaitu Tampilkan Antrian Aktif, sistem melakukan proses traversal pada queue. Proses traversal digunakan untuk menelusuri seluruh data antrian aktif dari bagian depan hingga bagian belakang. Seluruh data antrian yang masih aktif kemudian ditampilkan kepada pengguna, lalu program kembali ke menu utama.

Pada pilihan menu kelima, yaitu Tampilkan Riwayat, sistem melakukan proses traversal pada stack. Proses ini digunakan untuk menampilkan data antrian yang telah dipanggil sebelumnya. Riwayat ditampilkan berdasarkan prinsip kerja stack, yaitu data terakhir yang masuk akan berada pada posisi paling atas. Setelah riwayat ditampilkan, program kembali ke menu utama.

Pada pilihan menu keenam, yaitu Keluar, sistem akan menghentikan program. Jika pengguna memilih menu ini, maka alur program akan menuju simbol Selesai sebagai tanda bahwa program telah berakhir. Sementara itu, apabila pengguna memasukkan pilihan selain angka 1 sampai 6, sistem akan

menampilkan pesan bahwa pilihan tidak valid, kemudian mengarahkan kembali pengguna ke menu utama.

Berdasarkan flowchart tersebut, dapat disimpulkan bahwa sistem bekerja menggunakan alur perulangan menu. Program akan terus berjalan selama pengguna belum memilih menu keluar. Flowchart ini juga menunjukkan penerapan struktur data queue untuk mengelola antrian aktif, stack untuk menyimpan riwayat panggilan, serta algoritma linear search dan traversal untuk mendukung proses pencarian serta penampilan data.

3.3 Desain Struktur Data

Desain struktur data pada project ini disusun untuk mendukung proses pengelolaan antrian layanan secara efisien. Dalam implementasinya, sistem menggunakan tiga komponen utama, yaitu Node, QueueAntrian, dan StackRiwayat. Ketiga komponen tersebut saling berhubungan dalam menjalankan fungsi sistem, mulai dari penyimpanan data pelanggan, pengelolaan antrian aktif, hingga penyimpanan riwayat antrian yang telah dipanggil.

Node berfungsi sebagai unit dasar penyimpanan data. Setiap node menyimpan informasi penting berupa nomor antrian, nama pelanggan, keperluan, serta pointer next yang digunakan untuk menghubungkan node dengan node berikutnya. Dengan demikian, struktur node memungkinkan data disusun secara dinamis dalam bentuk linked list.

Selanjutnya, QueueAntrian digunakan untuk mengelola data antrian aktif. Struktur ini memiliki dua pointer utama, yaitu front dan rear. Pointer front menunjuk elemen paling depan dalam antrian, sedangkan rear menunjuk elemen paling belakang. Dengan rancangan tersebut, proses penambahan data antrian baru dapat dilakukan di bagian belakang menggunakan operasi

enqueue, sedangkan pemanggilan antrian dilakukan dari bagian depan menggunakan operasi dequeue. Prinsip kerja ini sesuai dengan konsep FIFO (First In First Out).

Selain itu, sistem juga menggunakan StackRiwayat untuk menyimpan data antrian yang telah dipanggil. Struktur ini memiliki satu pointer utama, yaitu top, yang menunjuk elemen teratas pada stack. Setiap data antrian yang selesai dipanggil akan dipindahkan ke dalam stack sebagai riwayat. Dengan demikian, riwayat antrian tersimpan berdasarkan prinsip LIFO (Last In First Out). Tabel berikut menunjukkan rancangan struktur data yang digunakan dalam sistem.

Tabel 2. Desain Struktur Data

Struktur	Atribut/Pointer	Fungsi
Node	nomor, nama, keperluan, next	Menyimpan satu data antrian dan menghubungkannya ke node berikutnya.
QueueAntrian	front dan rear	Menyimpan antrian aktif dengan prinsip FIFO.
StackRiwayat	top	Menyimpan riwayat antrian yang sudah dipanggil dengan prinsip LIFO.

Berdasarkan Tabel 2, dapat dipahami bahwa ketiga struktur data tersebut memiliki peran yang berbeda namun saling melengkapi. Node menjadi dasar pembentukan linked list, sedangkan QueueAntrian dan StackRiwayat memanfaatkan node tersebut untuk mengelola data sesuai kebutuhan sistem. Queue digunakan untuk memastikan urutan pelayanan berjalan secara adil berdasarkan urutan kedatangan, sementara stack digunakan untuk mencatat riwayat pemanggilan antrian.

Dengan desain ini, sistem mampu menangani proses penambahan antrian, pemanggilan antrian, pencarian data, penampilan antrian aktif, serta

penyimpanan riwayat secara terstruktur. Oleh karena itu, desain struktur data yang digunakan telah sesuai dengan kebutuhan fungsional program.

3.4 Desain Algoritma

Desain algoritma pada sistem antrian layanan disusun untuk menggambarkan langkah-langkah utama yang dijalankan program dalam mengelola data antrian. Algoritma yang digunakan disesuaikan dengan struktur data yang telah dirancang sebelumnya, yaitu queue untuk menyimpan antrian aktif dan stack untuk menyimpan riwayat antrian yang telah dipanggil.

Pada program ini, setiap fitur memiliki algoritma atau operasi utama yang berbeda sesuai dengan fungsinya. Fitur tambah antrian menggunakan operasi enqueue, fitur panggil antrian menggunakan operasi dequeue dan push, fitur cari antrian menggunakan algoritma linear search, sedangkan fitur tampilkan data menggunakan proses traversal pada queue maupun stack. Tabel berikut menunjukkan desain algoritma utama yang digunakan dalam sistem.

Tabel 3. Desain Algoritma

Fitur	Algoritma/Operasi	Penjelasan
Tambah Antrian	Enqueue	Membuat node baru lalu menambahkannya ke bagian belakang Queue.
Panggil Antrian	Dequeue + Push	Menghapus node dari bagian depan Queue lalu memasukkannya ke Stack riwayat.
Cari Antrian	Linear Search	Menelusuri Queue dari front sampai data nomor antrian ditemukan.
Tampilkan Antrian	Traversal Queue	Menampilkan seluruh node dari front sampai rear.

Tampilkan Riwayat	Traversal Stack	Menampilkan seluruh node dari top sampai node terakhir.
-------------------	-----------------	---

Berdasarkan tabel tersebut, algoritma enqueue digunakan pada fitur tambah antrian. Proses ini dimulai dengan pembuatan node baru yang berisi nomor antrian, nama, dan keperluan. Node baru tersebut kemudian ditempatkan pada bagian belakang queue melalui pointer rear. Dengan cara ini, data yang baru masuk akan selalu berada di urutan terakhir dalam antrian.

Pada fitur panggil antrian, algoritma yang digunakan adalah gabungan antara dequeue dan push. Operasi dequeue digunakan untuk mengambil data antrian yang berada di posisi paling depan, yaitu data yang ditunjuk oleh pointer front. Setelah data tersebut dipanggil, data tidak langsung dihapus dari sistem, tetapi dipindahkan ke dalam stack riwayat menggunakan operasi push. Hal ini bertujuan agar data antrian yang telah dipanggil tetap dapat ditampilkan sebagai riwayat layanan.

Pada fitur cari antrian, sistem menggunakan algoritma linear search. Algoritma ini bekerja dengan cara menelusuri data antrian satu per satu mulai dari bagian depan queue sampai data yang dicari ditemukan. Jika nomor antrian yang dimasukkan pengguna sesuai dengan salah satu data dalam queue, maka sistem akan menampilkan informasi antrian tersebut. Jika data tidak ditemukan sampai akhir queue, maka sistem akan menampilkan pesan bahwa data tidak tersedia.

Selanjutnya, fitur tampilkan antrian aktif menggunakan proses traversal queue. Proses traversal dilakukan dengan menelusuri seluruh node pada queue mulai dari front hingga rear. Setiap node yang dilalui akan ditampilkan kepada pengguna, sehingga pengguna dapat melihat seluruh data antrian yang masih aktif.

Fitur tampilkan riwayat menggunakan proses traversal stack. Proses ini dilakukan dengan menelusuri data mulai dari pointer top hingga node terakhir dalam stack. Dengan menggunakan stack, data riwayat yang terakhir masuk akan berada pada posisi paling atas, sesuai dengan prinsip LIFO (Last In First Out).

Secara umum, desain algoritma ini dibuat agar setiap fitur dalam program dapat berjalan sesuai fungsinya. Operasi enqueue, dequeue, dan push memiliki kompleksitas waktu $O(1)$ karena hanya melibatkan perubahan pointer pada bagian tertentu. Sementara itu, linear search dan traversal memiliki kompleksitas waktu $O(n)$ karena harus menelusuri data satu per satu. Dengan demikian, algoritma yang digunakan sudah sesuai dengan kebutuhan sistem antrian layanan yang bersifat sederhana, dinamis, dan interaktif.

IV. IMPLEMENTASI

4.1 Implementasi Program

Program Sistem Antrian Layanan diimplementasikan dalam dua bahasa pemrograman, yaitu C++ dan Python. Kedua versi program memiliki fitur dan alur kerja yang sama, yaitu menambah data antrian, memanggil antrian, mencari data antrian, menampilkan antrian aktif, menampilkan riwayat panggilan, serta keluar dari program.

Implementasi program dilakukan secara manual dengan membuat struktur data sendiri, yaitu Node, Queue, dan Stack. Dengan demikian, program tidak menggunakan library struktur data siap pakai. Pada program ini, Queue digunakan untuk menyimpan antrian aktif, sedangkan Stack digunakan untuk menyimpan riwayat antrian yang telah dipanggil.

4.2 . Implementasi Node

Node merupakan struktur dasar yang digunakan untuk menyimpan satu data antrian. Setiap node berisi beberapa atribut, yaitu nomor, nama, keperluan, dan next. Atribut nomor digunakan untuk menyimpan nomor antrian, nama digunakan untuk menyimpan nama pengguna layanan, keperluan digunakan untuk menyimpan tujuan atau keperluan pengguna, sedangkan next digunakan sebagai penghubung ke node berikutnya.

Pada program C++, node dibuat menggunakan struct Node. Di dalam struct tersebut terdapat atribut bertipe int, string, dan pointer Node* next. Pointer ini digunakan agar setiap data antrian dapat saling terhubung membentuk linked list. Pada program Python, node dibuat menggunakan class Node. Konstruktor __init__ digunakan untuk mengisi nilai awal dari setiap atribut. Atribut next diberi nilai awal None karena pada saat node baru dibuat, node tersebut belum terhubung dengan node lain.

```

6 struct Node {
7     int nomor;
8     string nama;
9     string keperluan;
10    Node* next;
11 };

```

Gambar 7. Implementasi Node pada C++

```

2 class Node:
3     def __init__(self, nomor, nama, keperluan):
4         self.nomor = nomor
5         self.nama = nama
6         self.keperluan = keperluan
7         self.next = None

```

Gambar 8. Implementasi Node pada Python

Pada C++, bagian struct Node digunakan untuk mendefinisikan bentuk data antrian. Setiap node menyimpan informasi antrian dan pointer menuju node berikutnya. Pada Python, konsep yang sama diterapkan melalui class Node. Perbedaannya hanya terletak pada sintaks bahasa pemrograman, sedangkan fungsi dan tujuan implementasinya tetap sama.

4.3 Implementasi Queue

Queue digunakan untuk menyimpan data antrian aktif. Struktur data ini bekerja berdasarkan prinsip FIFO (First In First Out), yaitu data yang

pertama masuk akan menjadi data pertama yang keluar. Prinsip ini sesuai dengan sistem antrian layanan karena pengguna yang mendaftar lebih awal harus dilayani lebih dahulu. Pada program ini, queue memiliki dua pointer utama, yaitu front dan rear. Pointer front menunjuk data paling depan dalam antrian, sedangkan pointer rear menunjuk data paling belakang. Dengan penggunaan dua pointer tersebut, proses penambahan dan penghapusan data dapat dilakukan secara lebih efisien.

```

14 class QueueAntrian {
15     private:
16         Node* front;
17         Node* rear;
18
19     public:
20         QueueAntrian() {
21             front = NULL;
22             rear = NULL;
23         }
24
25         // Mengecek apakah queue kosong
26         bool isEmpty() {
27             return front == NULL;
28         }

```

Gambar 9. Implementasi Class Queue pada C++

```

11 class QueueAntrian:
12     def __init__(self):
13         self.front = None
14         self.rear = None
15
16         # Mengecek apakah queue kosong
17     def is_empty(self):
18         return self.front is None

```

Gambar 10. Implementasi Class Queue pada Python

Pada bagian ini, program mendefinisikan class QueueAntrian. Di dalam class tersebut terdapat pointer front dan rear. Saat queue pertama kali dibuat, kedua pointer diberi nilai kosong, yaitu NULL pada C++ dan None pada Python. Hal ini menunjukkan bahwa pada awal program, belum ada data antrian yang tersimpan.

4.4 Implementasi Enqueue

Fungsi enqueue digunakan untuk menambahkan data baru ke dalam antrian. Data baru yang dimasukkan oleh pengguna akan dibuat menjadi node baru. Node tersebut kemudian dimasukkan ke bagian belakang queue. Jika queue masih kosong, maka pointer front dan rear akan menunjuk node baru yang sama. Namun, jika queue sudah memiliki data, maka node baru akan dihubungkan setelah pointer rear, kemudian pointer rear diperbarui agar menunjuk node baru tersebut.

```

31 void enqueue(int nomor, string nama, string keperluan) {
32     Node* newNode = new Node();
33     newNode->nomor = nomor;
34     newNode->nama = nama;
35     newNode->keperluan = keperluan;
36     newNode->next = NULL;
37
38     if (isEmpty()) {
39         front = rear = newNode;
40     } else {
41         rear->next = newNode;
42         rear = newNode;
43     }
44
45     cout << "Data antrian berhasil ditambahkan.\n";
46 }

```

Gambar 11. Implementasi Fungsi Enqueue pada C++

```

21 def enqueue(self, nomor, nama, keperluan):
22     new_node = Node(nomor, nama, keperluan)
23
24     if self.is_empty():
25         self.front = new_node
26         self.rear = new_node
27     else:
28         self.rear.next = new_node
29         self.rear = new_node
30
31     print("Data antrian berhasil ditambahkan.")

```

Gambar 12. Implementasi Fungsi Enqueue pada Python

Pada C++, node baru dibuat menggunakan perintah `new Node()`. Setelah itu, atribut nomor, nama, dan keperluan diisi berdasarkan input pengguna. Jika antrian kosong, maka front dan rear langsung diarahkan ke node baru. Jika antrian tidak kosong, maka `rear->next` diarahkan ke node baru dan pointer rear dipindahkan ke node baru. Pada Python, konsep yang

digunakan sama. Node baru dibuat dari class Node, kemudian diperiksa apakah queue kosong. Jika kosong, front dan rear menunjuk node baru. Jika tidak kosong, node baru dihubungkan setelah rear, lalu rear diperbarui.

4.5 Implementasi Dequeue

Fungsi dequeue digunakan untuk mengambil atau memanggil data antrian yang berada pada posisi paling depan. Fungsi ini sesuai dengan prinsip FIFO karena data yang pertama masuk akan dipanggil lebih dahulu. Jika queue kosong, fungsi akan mengembalikan nilai kosong. Namun, jika queue memiliki data, maka data pada pointer front akan disimpan sementara, kemudian pointer front dipindahkan ke node berikutnya. Jika setelah proses tersebut queue menjadi kosong, maka pointer rear juga dikosongkan.

```

49  Node* dequeue() {
50      if (isEmpty()) {
51          return NULL;
52      }
53
54      Node* temp = front;
55      front = front->next;
56
57      if (front == NULL) {
58          rear = NULL;
59      }
60
61      temp->next = NULL;
62      return temp;
63  }

```

Gambar 13. Implementasi Dequeue pada C++

```

34  def dequeue(self):
35      if self.is_empty():
36          return None
37
38      temp = self.front
39      self.front = self.front.next
40
41      if self.front is None:
42          self.rear = None
43
44      temp.next = None
45      return temp

```

Gambar 14. Implementasi Dequeue pada Python

Pada C++, pengecekan dilakukan terlebih dahulu menggunakan fungsi `isEmpty()`. Jika queue kosong, fungsi mengembalikan `NULL`. Jika tidak kosong, node pada bagian depan disimpan dalam variabel `temp`. Setelah itu, pointer `front` dipindahkan ke node berikutnya. Node yang telah dikeluarkan kemudian dikembalikan agar dapat diproses lebih lanjut, yaitu dimasukkan ke stack riwayat.

Pada Python, prosesnya sama. Jika queue kosong, fungsi mengembalikan `None`. Jika tidak kosong, data pada `front` disimpan ke variabel `temp`, lalu `front` dipindahkan ke node berikutnya. Data yang dikeluarkan kemudian dikembalikan ke bagian program utama.

4.6 Implementasi Tampilkan Antrian Aktif

Fungsi untuk menampilkan antrian aktif dilakukan dengan proses traversal pada queue. Traversal merupakan proses menelusuri seluruh node dari awal sampai akhir. Dalam queue, traversal dimulai dari pointer `front` sampai node terakhir. Jika queue kosong, program akan menampilkan pesan bahwa antrian masih kosong. Jika queue berisi data, program akan menampilkan nomor antrian, nama, dan keperluan dari setiap node.

```

66 void displayQueue() {
67     if (isEmpty()) {
68         cout << "Antrian masih kosong.\n";
69         return;
70     }
71
72     Node* current = front;
73
74     cout << "\n=== DAFTAR ANTRIAN AKTIF ===\n";
75     while (current != NULL) {
76         cout << "Nomor      : " << current->nomor << endl;
77         cout << "Nama        : " << current->nama << endl;
78         cout << "Keperluan  : " << current->keperluan << endl;
79         cout << "-----\n";
80         current = current->next;
81     }
82 }

```

Gambar 15. Implementasi Tampilkan Antrian Aktif pada C++

```

48     def display_queue(self):
49         if self.is_empty():
50             print("Antrian masih kosong.")
51             return
52
53         current = self.front
54
55         print("\n=== DAFTAR ANTRIAN AKTIF ===")
56         while current is not None:
57             print("Nomor      :", current.nomor)
58             print("Nama       :", current.nama)
59             print("Keperluan  :", current.keperluan)
60             print("-----")
61             current = current.next

```

Gambar 16. Implementasi Tampilkan Antrian Aktif pada Python

Pada bagian ini, variabel `current` digunakan untuk menelusuri data dari awal queue. Selama `current` belum bernilai kosong, program akan menampilkan isi node, kemudian memindahkan `current` ke node berikutnya. Proses ini terus berjalan sampai seluruh data antrian berhasil ditampilkan.

4.7 Implementasi Stack

Stack digunakan untuk menyimpan riwayat data antrian yang telah dipanggil. Struktur data stack bekerja berdasarkan prinsip LIFO (Last In First Out), yaitu data yang terakhir masuk akan berada pada posisi paling atas. Pada program ini, stack memiliki satu pointer utama, yaitu `top`. Pointer `top` menunjuk data teratas dalam stack. Setiap kali antrian dipanggil dari queue, data tersebut akan dimasukkan ke dalam stack sebagai riwayat panggilan.


```

113 class StackRiwayat {
114 private:
115     Node* top;
116
117 public:
118     StackRiwayat() {
119         top = NULL;
120     }
121
122     // Mengecek apakah stack kosong
123     bool isEmpty() {
124         return top == NULL;
125     }

```

Gambar 17. Implementasi Stack pada C++

```

88 class StackRiwayat:
89     def __init__(self):
90         self.top = None
91
92     # Mengecek apakah stack kosong
93     def is_empty(self):
94         return self.top is None

```

Gambar 18. Implementasi Stack pada Python

Pada C++, stack dibuat dalam class StackRiwayat dengan pointer top. Saat stack pertama kali dibuat, top diberi nilai NULL. Pada Python, top diberi nilai None. Nilai tersebut menunjukkan bahwa pada awal program, stack riwayat belum memiliki data.

4.8 Implementasi Push pada Stack

Fungsi push digunakan untuk menambahkan data ke bagian atas stack. Data yang dimasukkan ke stack berasal dari data antrian yang telah dipanggil melalui proses dequeue. Pada fungsi ini, node yang masuk akan diarahkan ke node yang sebelumnya berada di bagian atas stack. Setelah itu, pointer top diperbarui agar menunjuk node yang baru masuk.

```

128     void push(Node* data) {
129         data->next = top;
130         top = data;
131     }

```

Gambar 19. Implementasi Fungsi Push pada C++

```

97     def push(self, data):
98         data.next = self.top
99         self.top = data

```

Gambar 20. Implementasi Fungsi Push pada Python

Pada C++, perintah `data->next = top` digunakan untuk menghubungkan data baru dengan data yang sebelumnya berada di atas stack. Setelah itu, `top = data` digunakan untuk memperbarui posisi teratas stack. Pada Python, proses yang sama dilakukan dengan `data.next = self.top` dan `self.top = data`.

4.9 Implementasi Tampilkan Riwayat

Fungsi tampilkan riwayat digunakan untuk menampilkan seluruh data antrian yang telah dipanggil. Data ditampilkan dari bagian atas stack sampai node terakhir. Jika stack kosong, program akan menampilkan pesan bahwa riwayat panggilan masih kosong. Jika stack berisi data, program akan menampilkan nomor antrian, nama, dan keperluan dari setiap node yang tersimpan di dalam stack.

```

134 void displayStack() {
135     if (isEmpty()) {
136         cout << "Riwayat panggilan masih kosong.\n";
137         return;
138     }
139
140     Node* current = top;
141
142     cout << "\n=== RIWAYAT ANTRIAN DIPANGGIL ===\n";
143     while (current != NULL) {
144         cout << "Nomor      : " << current->nomor << endl;
145         cout << "Nama        : " << current->nama << endl;
146         cout << "Keperluan  : " << current->keperluan << endl;
147         cout << "-----\n";
148         current = current->next;
149     }
150 }

```

Gambar 21. Implementasi Fungsi Tampilkan Riwayat pada C++

```

102 def display_stack(self):
103     if self.is_empty():
104         print("Riwayat panggilan masih kosong.")
105         return
106
107     current = self.top
108
109     print("\n=== RIWAYAT ANTRIAN DIPANGGIL ===")
110     while current is not None:
111         print("Nomor      :", current.nomor)
112         print("Nama        :", current.nama)
113         print("Keperluan  :", current.keperluan)
114         print("-----")
115         current = current.next

```

Gambar 22. Implementasi Fungsi Tampilkan Riwayat pada Python

Pada bagian ini, variabel `current` digunakan untuk menelusuri stack mulai dari pointer `top`. Selama `current` belum kosong, program akan menampilkan data yang tersimpan, kemudian berpindah ke node berikutnya. Proses ini menunjukkan penerapan traversal pada struktur data stack.

4.10 Implementasi Searching

Fitur pencarian data antrian dilakukan menggunakan algoritma linear search. Algoritma ini bekerja dengan cara menelusuri data satu per satu dari bagian depan queue sampai data yang dicari ditemukan. Pada program ini, pengguna diminta memasukkan nomor antrian yang ingin dicari. Program

kemudian membandingkan nomor tersebut dengan atribut nomor pada setiap node. Jika nomor antrian ditemukan, program akan menampilkan data antrian. Jika tidak ditemukan sampai akhir queue, program akan menampilkan pesan bahwa data tidak ditemukan.

```

85 void searchByNomor(int nomorCari) {
86     if (isEmpty()) {
87         cout << "Antrian masih kosong.\n";
88         return;
89     }
90
91     Node* current = front;
92     bool found = false;
93
94     while (current != NULL) {
95         if (current->nomor == nomorCari) {
96             cout << "\nData antrian ditemukan:\n";
97             cout << "Nomor      : " << current->nomor << endl;
98             cout << "Nama       : " << current->nama << endl;
99             cout << "Keperluan : " << current->keperluan << endl;
100            found = true;
101            break;
102        }
103        current = current->next;
104    }
105
106    if (!found) {
107        cout << "Data antrian dengan nomor tersebut tidak ditemukan.\n";
108    }
109 }
110 };

```

Gambar 23 Implementasi Searching pada C++

```

64 def search_by_nomor(self, nomor_cari):
65     if self.is_empty():
66         print("Antrian masih kosong.")
67         return
68
69     current = self.front
70     found = False
71
72     while current is not None:
73         if current.nomor == nomor_cari:
74             print("\nData antrian ditemukan:")
75             print("Nomor      :", current.nomor)
76             print("Nama       :", current.nama)
77             print("Keperluan :", current.keperluan)
78             found = True
79             break
80
81         current = current.next
82
83     if not found:
84         print("Data antrian dengan nomor tersebut tidak ditemukan.")

```

Gambar 24. Implementasi Searching pada Python

Pada C++, proses pencarian dilakukan menggunakan pointer current yang dimulai dari front. Program menggunakan perulangan while untuk menelusuri setiap node. Jika current->nomor sama dengan nomor yang dicari, maka data akan ditampilkan dan variabel found diubah menjadi true. Pada Python, proses yang sama dilakukan dengan current = self.front dan perulangan while current is not None. Jika current.nomor sama dengan nomor_cari, maka data ditampilkan dan pencarian dihentikan menggunakan break.

4.11 Implementasi Menu Utama

Menu utama digunakan sebagai pusat interaksi antara pengguna dan program. Melalui menu utama, pengguna dapat memilih fitur yang ingin dijalankan. Program menyediakan enam pilihan menu, yaitu tambah antrian, panggil antrian, cari antrian, tampilkan antrian aktif, tampilkan riwayat panggilan, dan keluar.

Pada program C++, menu utama dibuat menggunakan perulangan do while. Perulangan ini membuat program terus berjalan selama pengguna belum memilih menu keluar. Pada program Python, menu utama dibuat menggunakan perulangan while True, kemudian dihentikan menggunakan perintah break ketika pengguna memilih menu keluar.

```

153 int main() {
154     QueueAntrian queue;
155     StackRiwayat stack;
156
157     int pilihan;
158     int nomorOtomatis = 1;
159
160     do {
161         cout << "\n===== SISTEM ANTRIAN LAYANAN =====\n";
162         cout << "1. Tambah Antrian\n";
163         cout << "2. Panggil Antrian\n";
164         cout << "3. Cari Antrian\n";
165         cout << "4. Tampilkan Antrian Aktif\n";
166         cout << "5. Tampilkan Riwayat Panggilan\n";
167         cout << "6. Keluar\n";
168         cout << "Pilih menu: ";
169         cin >> pilihan;
170         cin.ignore();

```

Gambar 25. Implementasi Menu Utama pada C++

```

118 def main():
119     queue = QueueAntrian()
120     stack = StackRiwayat()
121
122     nomor_otomatis = 1
123
124     while True:
125         print("\n===== SISTEM ANTRIAN LAYANAN =====")
126         print("1. Tambah Antrian")
127         print("2. Panggil Antrian")
128         print("3. Cari Antrian")
129         print("4. Tampilkan Antrian Aktif")
130         print("5. Tampilkan Riwayat Panggilan")
131         print("6. Keluar")
132
133         pilihan = input("Pilih menu: ")

```

Gambar 26. Implementasi Menu Utama pada Python

Pada bagian ini, objek QueueAntrian dan StackRiwayat dibuat terlebih dahulu. Selain itu, terdapat variabel nomorOtomatis atau nomor_otomatis yang digunakan untuk memberikan nomor antrian secara otomatis kepada pengguna. Program kemudian menampilkan daftar menu dan menerima input pilihan dari pengguna.

4.12 Implementasi Menu Tambah Antrian

Menu tambah antrian digunakan untuk memasukkan data pengguna baru ke dalam queue. Pengguna diminta memasukkan nama dan keperluan.

Setelah itu, sistem memanggil fungsi enqueue untuk menambahkan data ke bagian belakang antrian.

```

172         if (pilihan == 1) {
173             string nama, keperluan;
174
175             cout << "Masukkan nama: ";
176             getline(cin, nama);
177
178             cout << "Masukkan keperluan: ";
179             getline(cin, keperluan);
180
181             queue.enqueue(nomorOtomatis, nama, keperluan);
182
183             cout << "Nomor antrian Anda: " << nomorOtomatis << endl;
184             nomorOtomatis++;

```

Gambar 27. Implementasi Menu Tambah Antrian pada C++

```

135         if pilihan == "1":
136             nama = input("Masukkan nama: ")
137             keperluan = input("Masukkan keperluan: ")
138
139             queue.enqueue(nomor_otomatis, nama, keperluan)
140
141             print("Nomor antrian Anda:", nomor_otomatis)
142             nomor_otomatis += 1

```

Gambar 28. Implementasi Menu Tambah Antrian pada Python

Pada C++, input nama dan keperluan menggunakan getline agar program dapat menerima teks yang mengandung spasi. Setelah data dimasukkan, fungsi queue.enqueue() dipanggil dengan parameter nomor antrian, nama, dan keperluan. Nomor antrian kemudian dinaikkan satu angka agar data berikutnya memiliki nomor yang berbeda. Pada Python, input dilakukan menggunakan fungsi input(). Setelah data dimasukkan, sistem memanggil queue.enqueue(), lalu menampilkan nomor antrian kepada pengguna.

4.13 Implementasi Menu Panggil Antrian

Menu panggil antrian digunakan untuk mengambil data antrian paling depan dari queue. Data yang berhasil dipanggil kemudian dimasukkan ke stack riwayat agar dapat ditampilkan kembali pada menu riwayat panggilan.

```

186         } else if (pilihan == 2) {
187             Node* dipanggil = queue.dequeue();
188
189             if (dipanggil == NULL) {
190                 cout << "Tidak ada antrian yang bisa dipanggil.\n";
191             } else {
192                 cout << "\nAntrian dipanggil:\n";
193                 cout << "Nomor      : " << dipanggil->nomor << endl;
194                 cout << "Nama       : " << dipanggil->nama << endl;
195                 cout << "Keperluan : " << dipanggil->keperluan << endl;
196
197                 stack.push(dipanggil);
198             }

```

Gambar 29. Implementasi Menu Panggil Antrian pada C++

```

145         dipanggil = queue.dequeue()
146
147         if dipanggil is None:
148             print("Tidak ada antrian yang bisa dipanggil.")
149         else:
150             print("\nAntrian dipanggil:")
151             print("Nomor      :", dipanggil.nomor)
152             print("Nama       :", dipanggil.nama)
153             print("Keperluan :", dipanggil.keperluan)
154
155             stack.push(dipanggil)

```

Gambar 30. Implementasi Menu Panggil Antrian pada Python

Pada bagian ini, program memanggil fungsi dequeue. Jika hasilnya kosong, berarti tidak ada antrian yang dapat dipanggil. Namun, jika terdapat data, maka program menampilkan data antrian tersebut dan menyimpannya ke stack riwayat menggunakan fungsi push.

4.14 Implementasi Menu Cari Antrian

Menu cari antrian digunakan untuk mencari data berdasarkan nomor antrian. Pengguna memasukkan nomor antrian yang ingin dicari, kemudian program memanggil fungsi pencarian.

```

200         } else if (pilihan == 3) {
201             int nomorCari;
202
203             cout << "Masukkan nomor antrian yang dicari: ";
204             cin >> nomorCari;
205
206             queue.searchByNomor(nomorCari);

```

Gambar 31. Implementasi Menu Cari Antrian pada C++

```

157         elif pilihan == "3":
158             nomor_cari = int(input("Masukkan nomor antrian yang dicari: "))
159             queue.search_by_nomor(nomor_cari)

```

Gambar 32. Implementasi Menu Cari Antrian pada Python

Pada C++, nomor antrian yang dicari disimpan dalam variabel nomorCari, kemudian dikirim ke fungsi searchByNomor. Pada Python, nomor yang dicari disimpan dalam variabel nomor_cari, kemudian dikirim ke fungsi search_by_nomor.

4.15 Implementasi Menu Tampilkan Data dan Keluar

Menu tampilkan data terdiri dari dua bagian, yaitu menampilkan antrian aktif dan menampilkan riwayat panggilan. Jika pengguna memilih menu 4, program akan memanggil fungsi untuk menampilkan queue. Jika pengguna memilih menu 5, program akan memanggil fungsi untuk menampilkan stack. Jika pengguna memilih menu 6, program akan berhenti. Jika pilihan tidak sesuai dengan menu yang tersedia, program akan menampilkan pesan bahwa pilihan tidak valid.

```

208         } else if (pilihan == 4) {
209             queue.displayQueue();
210         }
211         } else if (pilihan == 5) {
212             stack.displayStack();
213         }
214         } else if (pilihan == 6) {
215             cout << "Program selesai.\n";
216         }
217         } else {
218             cout << "Pilihan tidak valid.\n";
219         }
220     }
221     } while (pilihan != 6);
222
223     return 0;
224 }

```

Gambar 33. Implementasi Menu Tampilkan Data dan Keluar pada C++

```

167         elif pilihan == "6":
168             print("Program selesai.")
169             break
170
171         else:
172             print("Pilihan tidak valid.")
173
174
175     main()

```

Gambar 34. Implementasi Menu Tampilkan Data dan Keluar pada Python

Pada bagian ini, program menghubungkan pilihan pengguna dengan fungsi yang sesuai. Menu 4 menjalankan fungsi `displayQueue` atau `display_queue`, menu 5 menjalankan fungsi `displayStack` atau `display_stack`, sedangkan menu 6 digunakan untuk mengakhiri program. Apabila pengguna memasukkan pilihan selain menu yang tersedia, program akan menampilkan pesan pilihan tidak valid.

V. KESIMPULAN

5.1 Hasil Akhir

Project Sistem Antrian Layanan berhasil dibuat dalam dua bahasa pemrograman, yaitu C++ dan Python. Program dapat dijalankan melalui terminal atau command prompt dan menyediakan fitur tambah antrian, panggil antrian, cari antrian, tampilkan antrian aktif, tampilkan riwayat panggilan, dan keluar dari program.

Program ini berhasil menerapkan struktur data Queue untuk mengelola antrian aktif dan Stack untuk menyimpan riwayat panggilan. Selain itu, program juga menerapkan algoritma linear search untuk mencari data antrian dan traversal untuk menampilkan seluruh data.

5.2 Kekurangan

Kekurangan dari project ini adalah sebagai berikut:

- a) Data belum tersimpan secara permanen karena belum menggunakan file atau database.
- b) Program masih berbasis terminal dan belum memiliki tampilan GUI.
- c) Pencarian hanya dapat dilakukan berdasarkan nomor antrian.

Belum terdapat fitur edit data antrian atau pembatalan antrian.

5.3 Saran Pengembangan

Saran pengembangan untuk project ini adalah sebagai berikut:

- a) Menambahkan penyimpanan data ke file agar data tetap ada setelah program ditutup.
- b) Menambahkan fitur edit dan hapus data antrian tertentu.
- c) Menambahkan pencarian berdasarkan nama atau keperluan.
- d) Mengembangkan program menjadi aplikasi GUI atau berbasis web.

Menambahkan laporan jumlah antrian yang sudah dilayani dan yang masih menunggu.

5.4 Kesimpulan Umum

Berdasarkan hasil implementasi, dapat disimpulkan bahwa Queue dan Stack dapat digunakan untuk menyelesaikan permasalahan sistem antrian layanan. Queue sesuai untuk mengatur antrian aktif karena mengikuti prinsip FIFO, sedangkan Stack sesuai untuk menyimpan riwayat panggilan karena mengikuti prinsip LIFO. Dengan implementasi manual menggunakan Node/Linked List, project ini menunjukkan pemahaman terhadap konsep dasar struktur data, algoritma searching, traversal, dan analisis kompleksitas.

DAFTAR PUSTAKA

Morin, P. (2013). *Open Data Structures: An Introduction*. Athabasca University Press. Available at:

https://www.aupress.ca/app/uploads/120226_99Z_Morin_2013-Open_Data_Structures.pdf.

Digunakan sebagai referensi dasar teori struktur data, queue, stack, linked list, linear search, traversal, dan kompleksitas algoritma. Bagian yang digunakan: halaman buku 5–6, 12–14, 19–20, 63–66, dan 108.